

YZ 203 - Week 2: Lab / Practice Session Notes

Lab Objectives

By the end of this session, the student will be able to:

- Build and run standalone examples for all nine components covered this week: JCheckBox, JRadioButton, JComboBox, JList, JTextArea, JScrollPane, JSlider, JSpinner, JProgressBar
- Choose the right listener interface for a given component (ActionListener, ItemListener, ListSelectionListener, ChangeListener)
- Combine multiple selection components into one working panel, and multiple value components into one working HUD
- Avoid the most common mistakes for each component (missing ButtonGroup, missing JScrollPane, unguarded drag events, wrong cast on JSpinner's value)

Duration: About 2 class hours (the hands-on block after the theory lecture) **Required materials:** JDK and NetBeans from Week 1 — no new setup needed

Exercise 1: JCheckBox

Goal: Toggle an independent setting and read its state with `ItemListener`.

```
import javax.swing.*;

public class CheckBoxDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Sound Settings");
        JPanel panel = new JPanel();

        JCheckBox soundBox = new JCheckBox("Sound on", true);
        JLabel status = new JLabel("Sound: ON");

        soundBox.addItemListener(e -> {
            boolean on = soundBox.isSelected();
            status.setText(on ? "Sound: ON" : "Sound: OFF");
        });

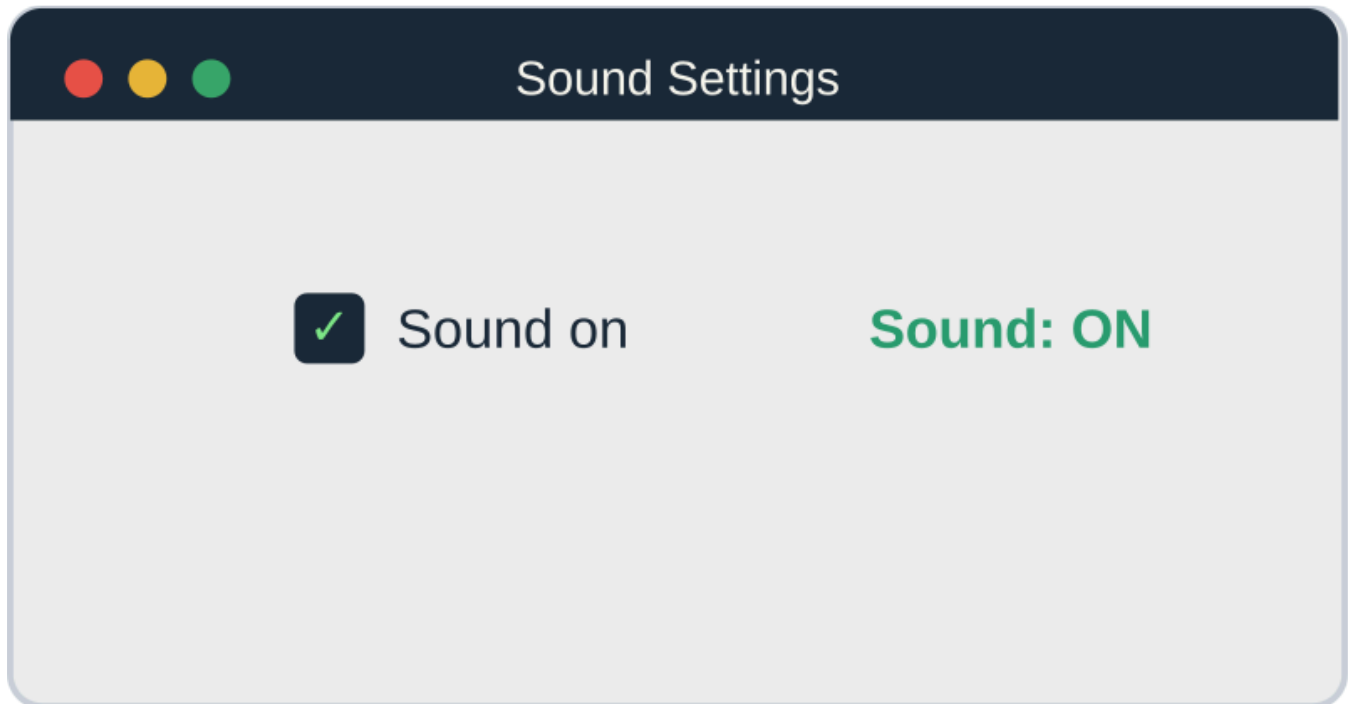
        panel.add(soundBox);
        panel.add(status);
        frame.add(panel);
        frame.setSize(300, 150);
    }
}
```

```

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

Expected output:



Try it: uncheck the box and confirm the label switches to "Sound: OFF" immediately — no button click needed, `ItemListener` fires on the check itself.

Exercise 2: JRadioButton + ButtonGroup

Goal: Offer mutually exclusive choices, and see what breaks without a group.

```

import javax.swing.*;

public class RadioButtonDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Difficulty");
        JPanel panel = new JPanel();

        JRadioButton easy = new JRadioButton("Easy");
        JRadioButton medium = new JRadioButton("Medium", true);
        JRadioButton hard = new JRadioButton("Hard");

        ButtonGroup group = new ButtonGroup();
        group.add(easy);
        group.add(medium);
        group.add(hard);
    }
}

```

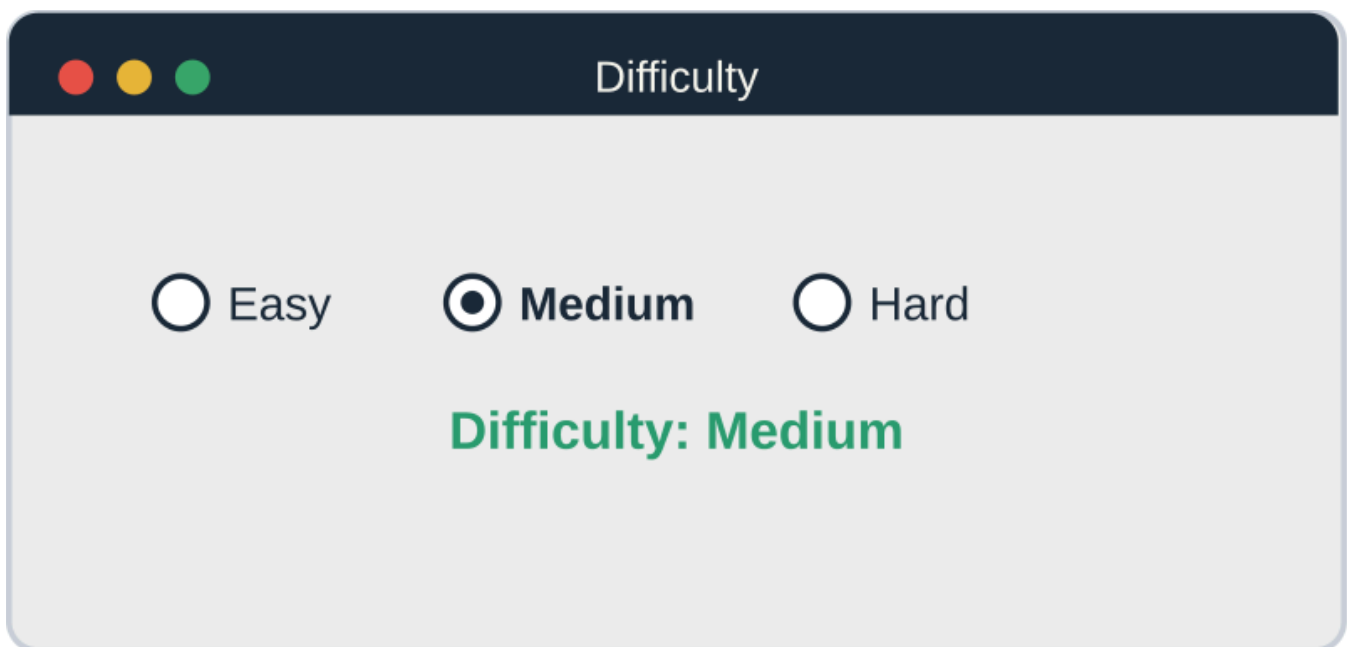
```

JLabel status = new JLabel("Difficulty: Medium");
easy.addActionListener(e -> status.setText("Difficulty: Easy"));
medium.addActionListener(e -> status.setText("Difficulty: Medium"));
hard.addActionListener(e -> status.setText("Difficulty: Hard"));

panel.add(easy); panel.add(medium); panel.add(hard);
panel.add(status);
frame.add(panel);
frame.setSize(360, 150);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Expected output:



Try it: comment out the three `group.add(...)` lines, run again, and click all three buttons at once — without `ButtonGroup`, all three stay checked simultaneously.

Exercise 3: JComboBox

Goal: Pick one value from a short dropdown list using `ItemListener`.

```

import javax.swing.*;
import java.awt.event.ItemEvent;

public class ComboBoxDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Character Color");
    }
}

```

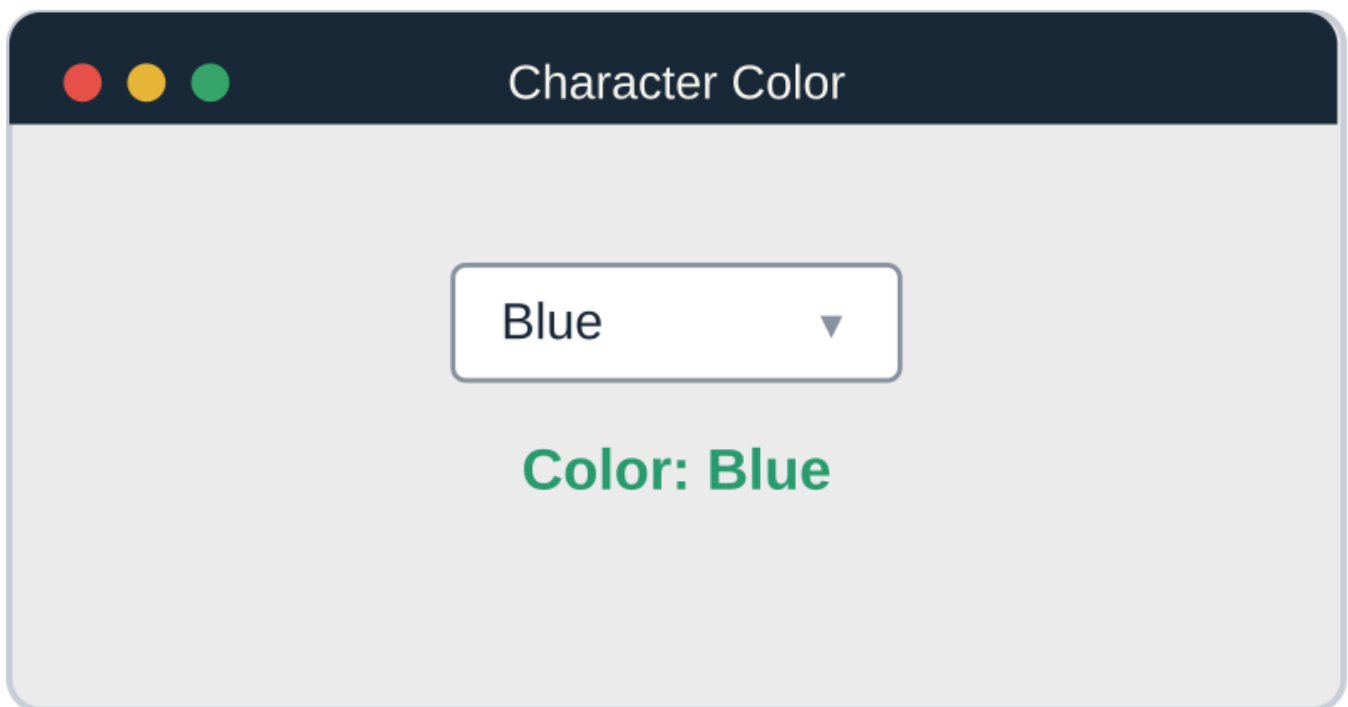
```
JPanel panel = new JPanel();

String[] colors = {"Red", "Blue", "Green", "Yellow"};
JComboBox<String> colorBox = new JComboBox<>(colors);
JLabel status = new JLabel("Color: Red");

colorBox.addItemListener(e -> {
    if (e.getStateChange() == ItemEvent.SELECTED) {
        status.setText("Color: " + colorBox.getSelectedItem());
    }
});

panel.add(colorBox);
panel.add(status);
frame.add(panel);
frame.setSize(300, 150);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
```

Expected output:



Try it: remove the `if (e.getStateChange() == ItemEvent.SELECTED)` check and watch the label update twice per selection — once for the item being deselected, once for the one being selected.

Exercise 4: JList

Goal: Pick a player from a scoreboard, guarding against mid-drag noise.

```
import javax.swing.*;

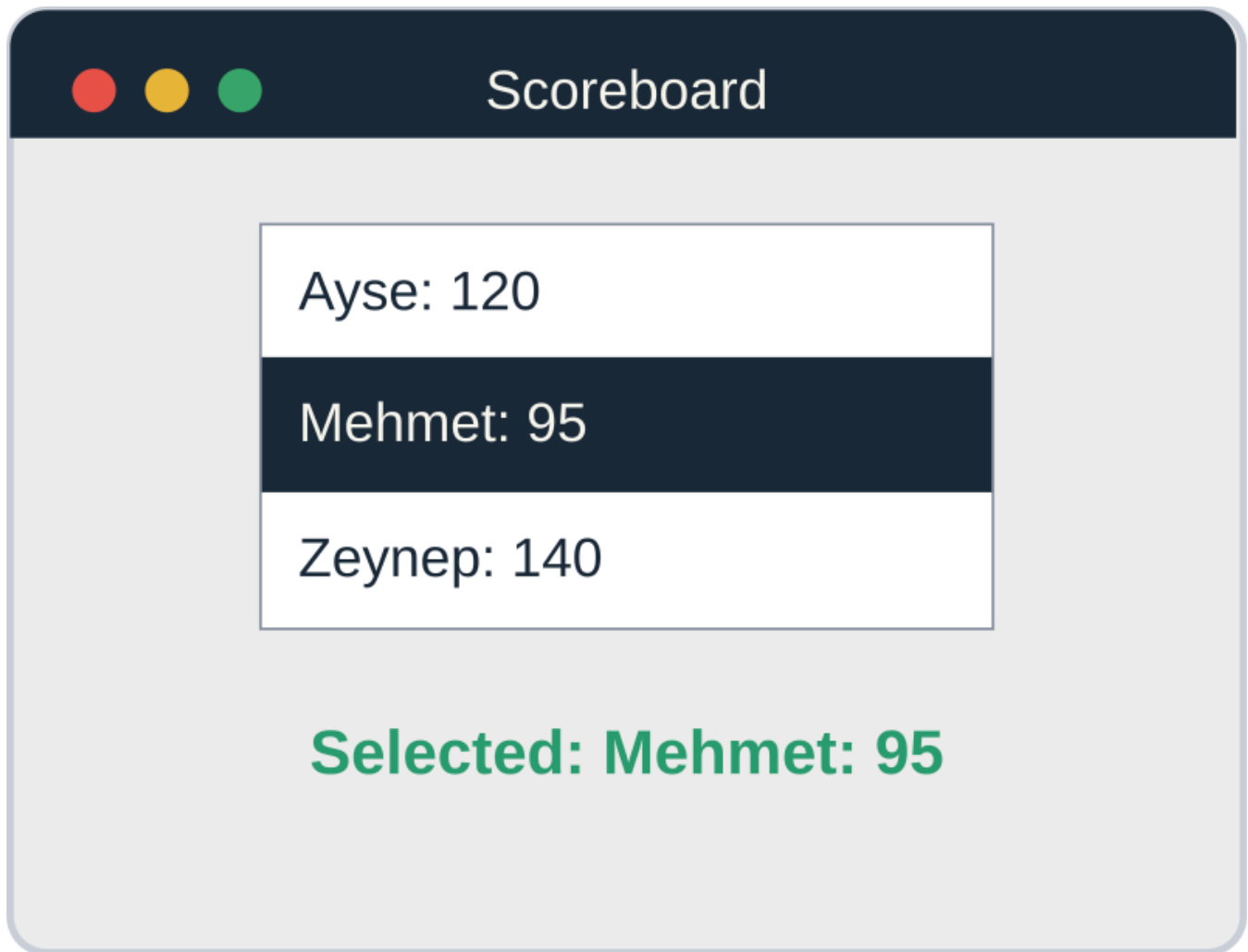
public class ListDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Scoreboard");
        JPanel panel = new JPanel();

        String[] players = {"Ayse: 120", "Mehmet: 95", "Zeynep: 140"};
        JList<String> playerList = new JList<>(players);
        JLabel status = new JLabel("Pick a player");

        playerList.addListSelectionListener(e -> {
            if (!e.getValueIsAdjusting()) {
                status.setText("Selected: " + playerList.getSelectedValue());
            }
        });

        panel.add(playerList);
        panel.add(status);
        frame.add(panel);
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Expected output:



Try it: remove the `!e.getValueIsAdjusting()` guard, click-and-drag across two rows, and count how many times the label updates during that single drag.

Exercise 5: Combining All Four into a Game Settings Panel

Goal: Read four different components through one shared `ActionListener`.

```
import javax.swing.*;

public class GameSettings {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Game Settings");
        JPanel panel = new JPanel();

        JCheckBox soundBox = new JCheckBox("Sound", true);

        JRadioButton easy = new JRadioButton("Easy");
        JRadioButton medium = new JRadioButton("Medium", true);
        JRadioButton hard = new JRadioButton("Hard");
        ButtonGroup diff = new ButtonGroup();
```

```

diff.add(easy); diff.add(medium); diff.add(hard);

String[] colors = {"Red", "Blue", "Green"};
JComboBox<String> colorBox = new JComboBox<>(colors);

JButton startBtn = new JButton("Start");
JLabel summary = new JLabel(" ");

startBtn.addActionListener(e -> {
    String diffText = easy.isSelected() ? "Easy" :
        hard.isSelected() ? "Hard" : "Medium";
    summary.setText("Sound=" + soundBox.isSelected()
        + ", Diff=" + diffText
        + ", Color=" + colorBox.getSelectedItem());
});

panel.add(soundBox);
panel.add(easy); panel.add(medium); panel.add(hard);
panel.add(colorBox);
panel.add(startBtn); panel.add(summary);

frame.add(panel);
frame.setSize(420, 220);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Expected output:

Try it: add a `JList` of saved profiles to the panel and include the selected profile name in the summary label too.

Exercise 6: JTextArea

Goal: Build a running log that appends lines instead of replacing them.

```
import javax.swing.*;

public class TextAreaDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Game Log");
        JPanel panel = new JPanel();

        JTextArea log = new JTextArea(6, 22);
        log.setEditable(false);
        log.setLineWrap(true);

        JButton addLine = new JButton("Egg caught!");
        addLine.addActionListener(e ->
            log.append("Egg caught!\n"));

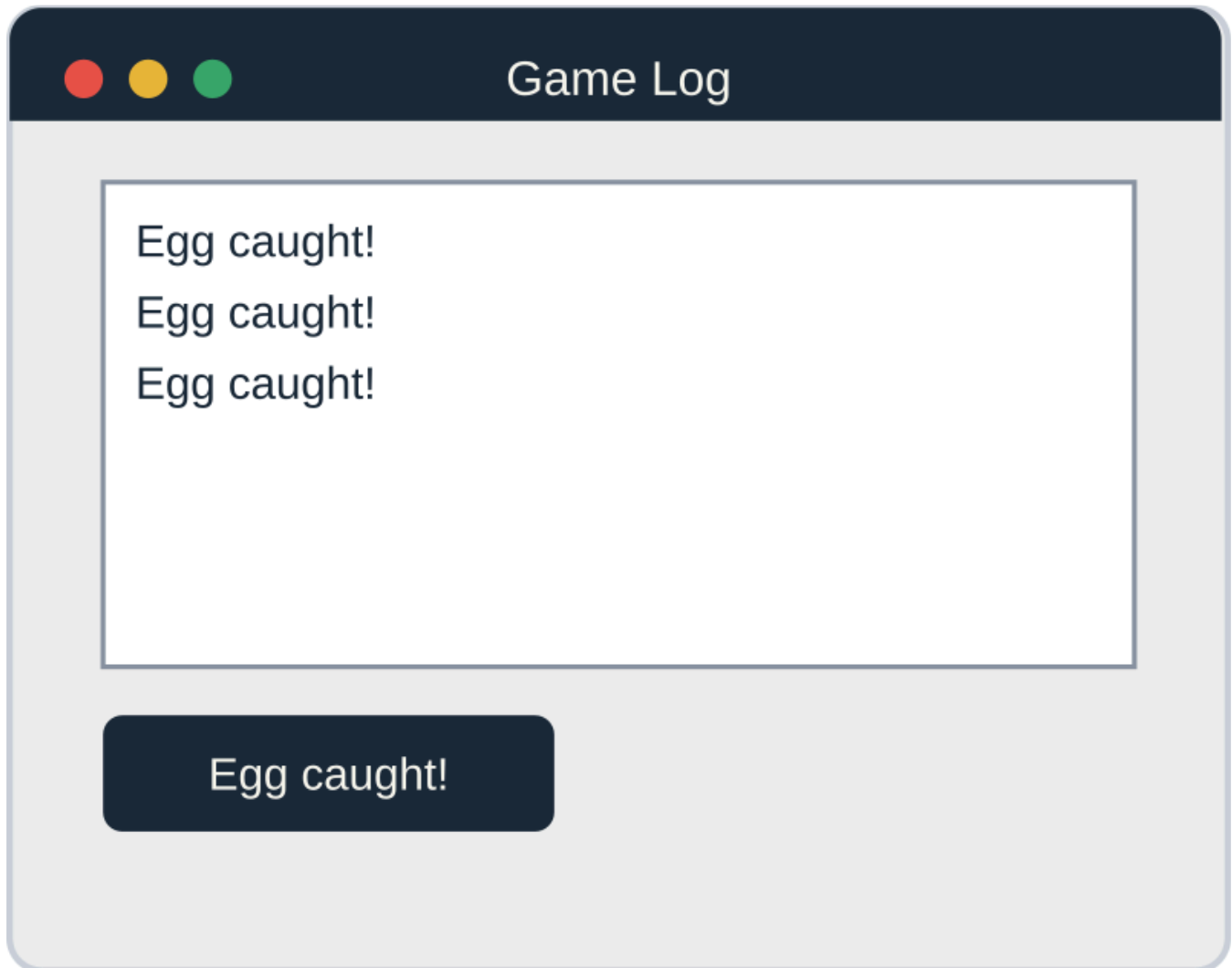
        panel.add(new JScrollPane(log));
        panel.add(addLine);
        frame.add(panel);
        frame.setSize(320, 260);
    }
}
```

```

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

Expected output:



Try it: replace `log.append(...)` with `log.setText(...)` and notice every previous line disappears — `setText()` replaces, `append()` adds.

Exercise 7: JScrollPane

Goal: Wrap a long `JList` so it scrolls instead of overflowing.

```

import javax.swing.*;
import java.awt.Dimension;

public class ScrollPaneDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Full Scoreboard");
    }
}

```

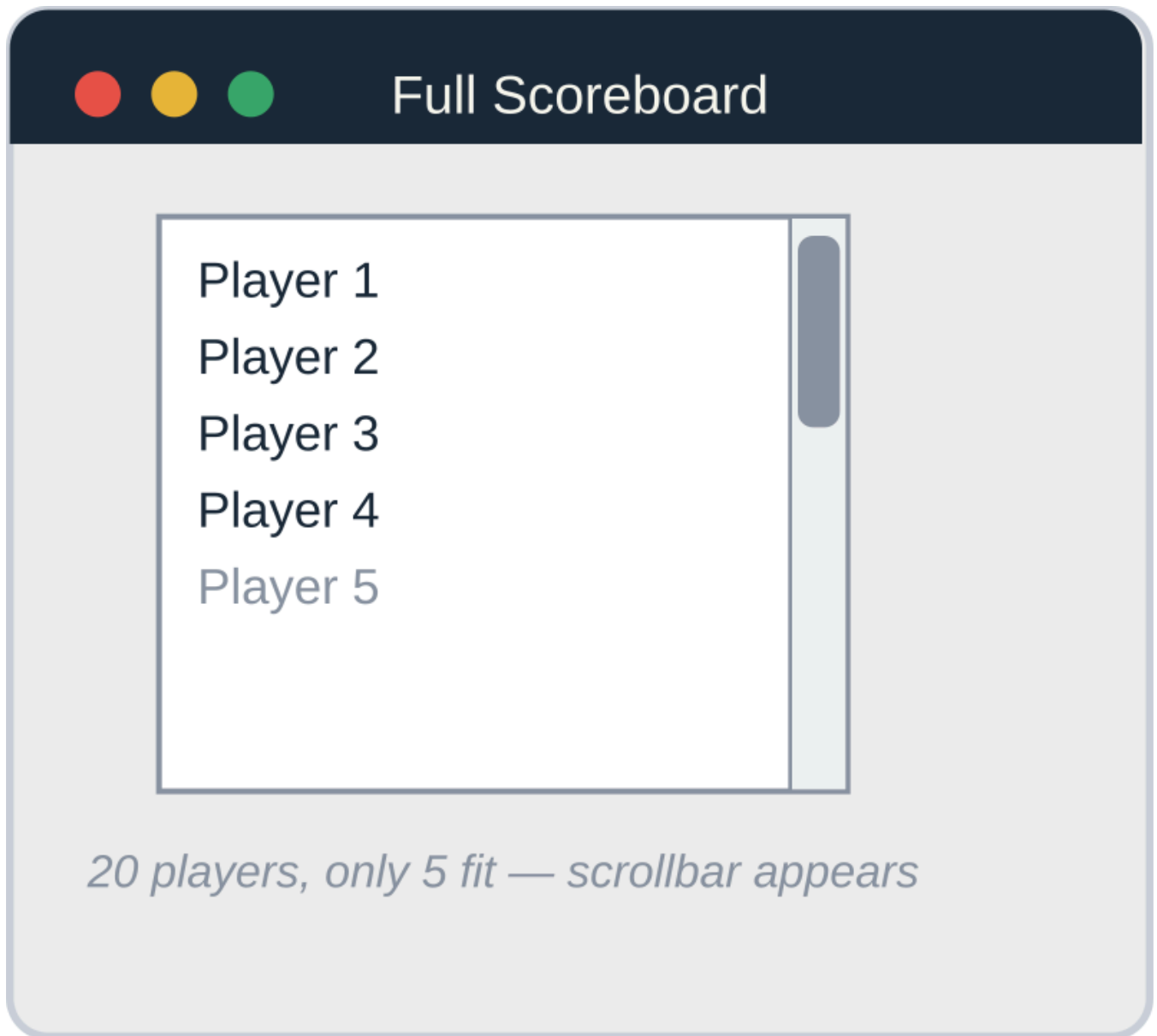
```
JPanel panel = new JPanel();

String[] players = new String[20];
for (int i = 0; i < 20; i++)
    players[i] = "Player " + (i + 1);

JList<String> list = new JList<>(players);
JScrollPane scrollPane = new JScrollPane(list);
scrollPane.setPreferredSize(new Dimension(200, 150));

panel.add(scrollPane);
frame.add(panel);
frame.setSize(280, 250);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
    }
}
```

Expected output:



Try it: remove `scrollPane.setPreferredSize(...)` and see the scroll pane shrink to a nearly invisible sliver — it never guesses a sensible size on its own.

Exercise 8: JSlider

Goal: Drag to set a volume value, guarding against constant mid-drag firing.

```
import javax.swing.*;

public class SliderDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Volume");
        JPanel panel = new JPanel();

        JSlider volume = new JSlider(0, 100, 70);
        volume.setMajorTickSpacing(25);
```

```

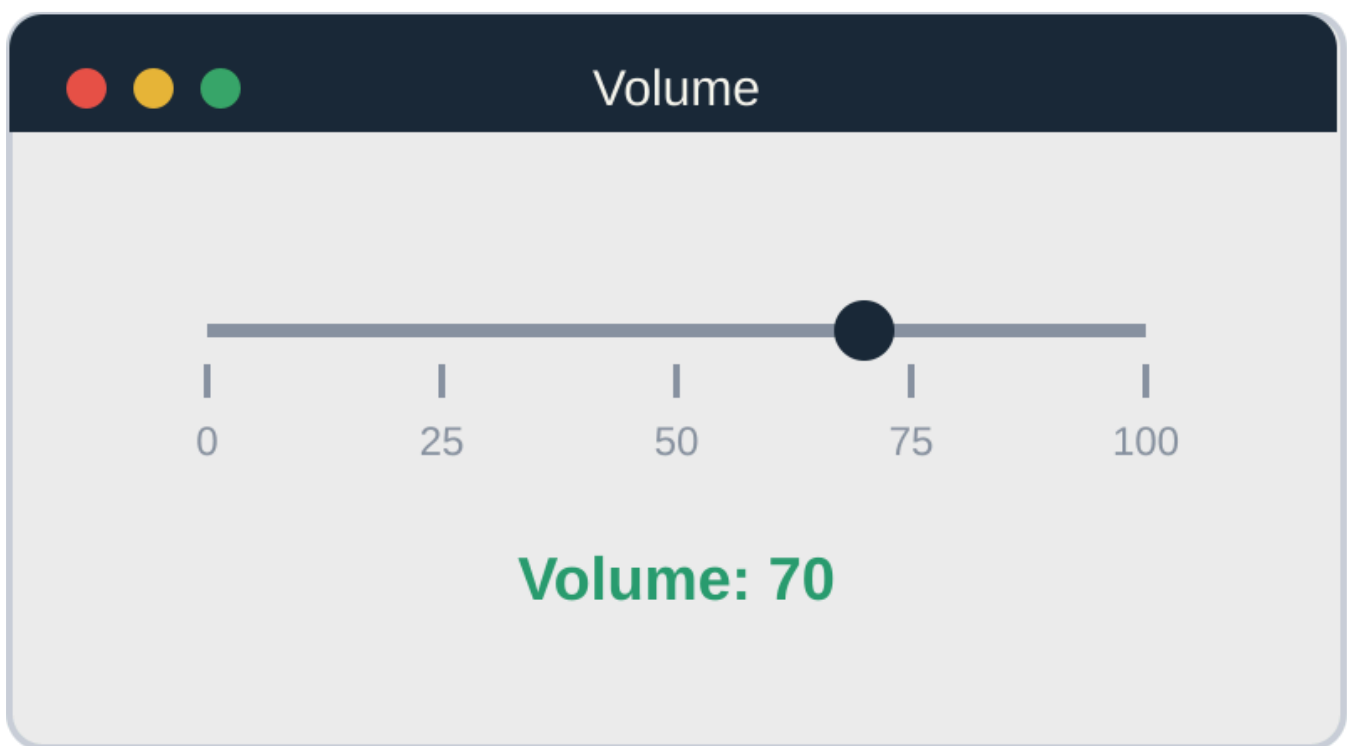
        volume.setPaintTicks(true);
        volume.setPaintLabels(true);

        JLabel status = new JLabel("Volume: 70");
        volume.addChangeListener(e -> {
            if (!volume.getValueIsAdjusting()) {
                status.setText("Volume: " + volume.getValue());
            }
        });

        panel.add(volume); panel.add(status);
        frame.add(panel);
        frame.setSize(340, 170);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

Expected output:



Try it: remove `setPaintTicks(true)` and confirm the tick spacing you set has no visible effect without it.

Exercise 9: JSpinner

Goal: Pick a precise number of starting lives with a `SpinnerNumberModel`.

```
import javax.swing.*;

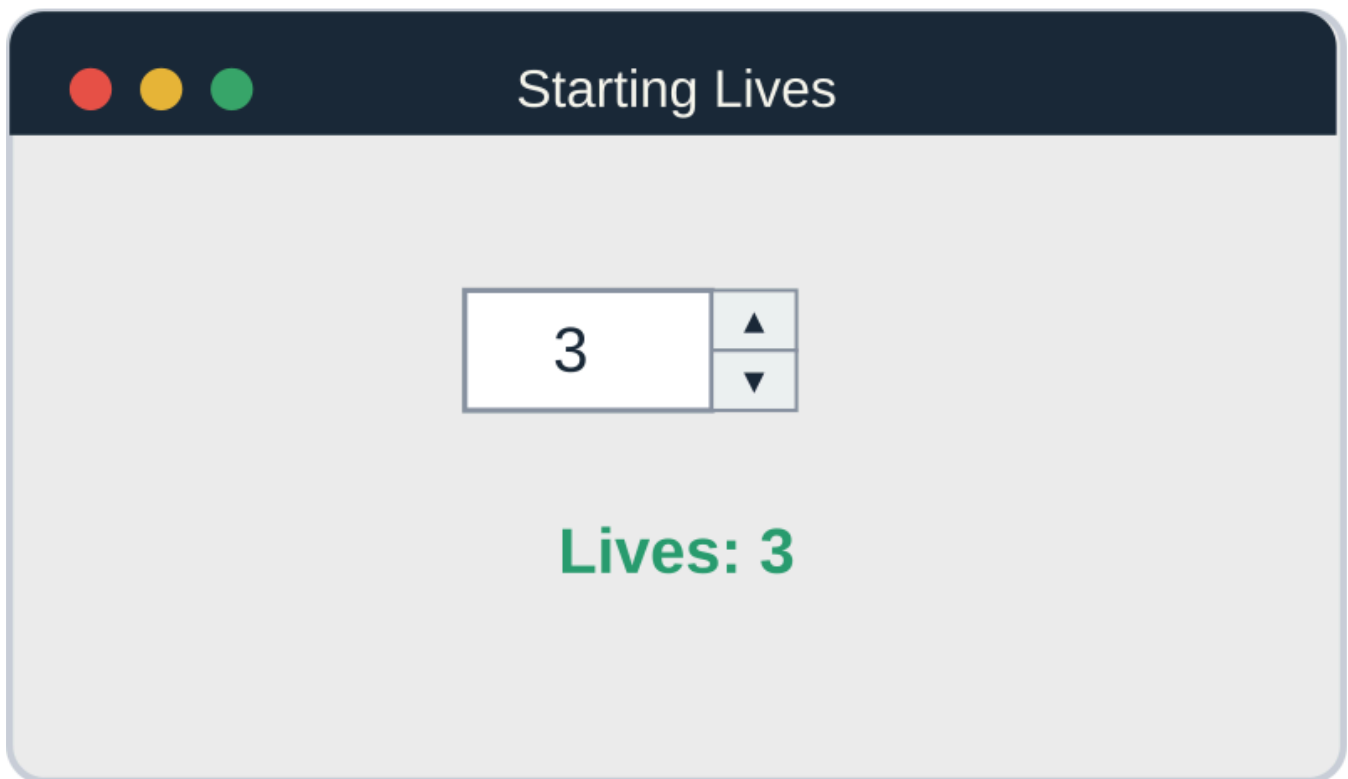
public class SpinnerDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Starting Lives");
        JPanel panel = new JPanel();

        SpinnerNumberModel model = new SpinnerNumberModel(3, 1, 9, 1);
        JSpinner livesSpinner = new JSpinner(model);

        JLabel status = new JLabel("Lives: 3");
        livesSpinner.addChangeListener(e -> {
            int lives = (Integer) livesSpinner.getValue();
            status.setText("Lives: " + lives);
        });

        panel.add(livesSpinner); panel.add(status);
        frame.add(panel);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Expected output:



Try it: change `int lives = (Integer) livesSpinner.getValue();` to `int lives = livesSpinner.getValue();` without the cast and read the compiler error carefully — `getValue()` returns a plain Object.

Exercise 10: JProgressBar

Goal: Drain a health bar on each button click, with the percentage painted on it.

```
import javax.swing.*;

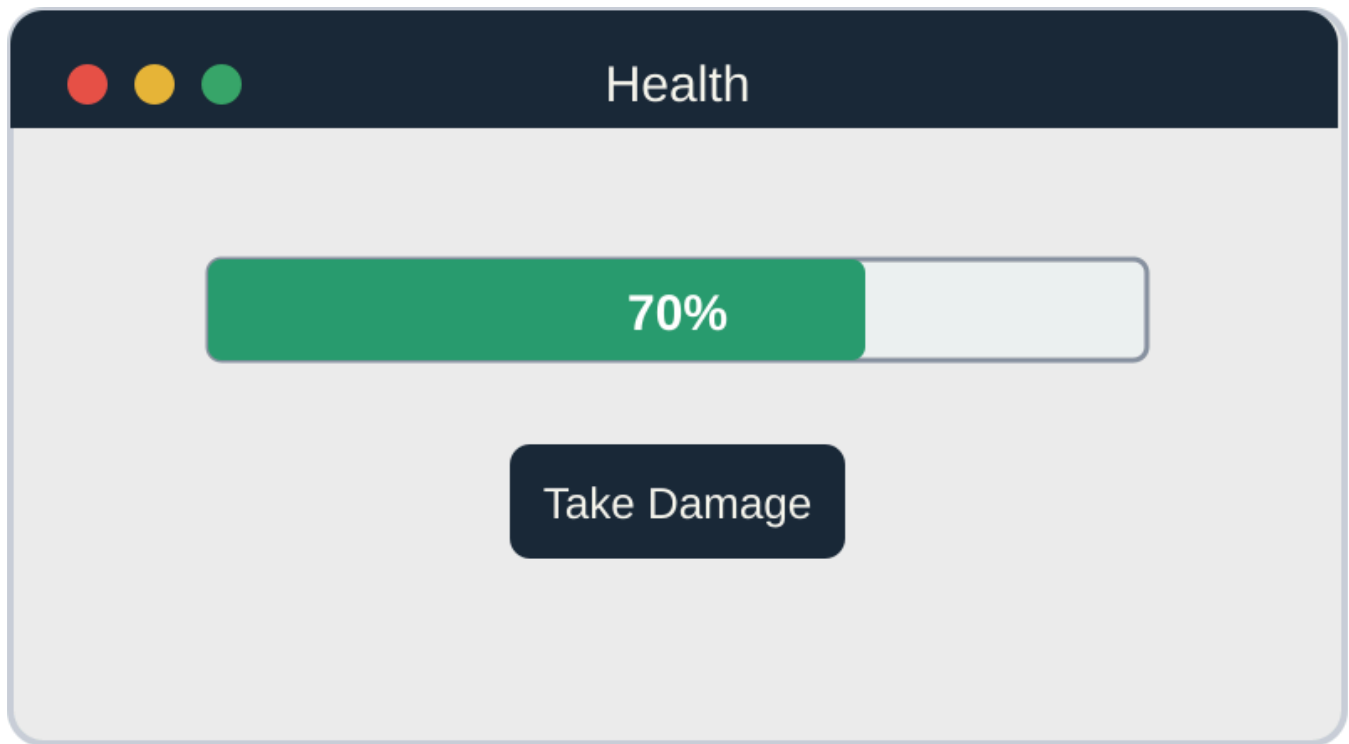
public class ProgressBarDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Health");
        JPanel panel = new JPanel();

        JProgressBar health = new JProgressBar(0, 100);
        health.setValue(100);
        health.setStringPainted(true);

        JButton hit = new JButton("Take Damage");
        hit.addActionListener(e -> {
            int newValue = health.getValue() - 15;
            health.setValue(Math.max(0, newValue));
        });

        panel.add(health); panel.add(hit);
        frame.add(panel);
        frame.setSize(320, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Expected output:



Try it: remove `setStringPainted(true)` and confirm the bar still fills correctly — it just no longer shows the percentage text.

Exercise 11: Combining Value Components into a Game HUD

Goal: Wire a slider, a spinner, a progress bar, and a log together behind one button.

```
import javax.swing.*;

public class GameHud {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Game HUD");
        JPanel panel = new JPanel();

        JSlider volume = new JSlider(0, 100, 70);
        JSpinner lives = new JSpinner(
            new SpinnerNumberModel(3, 1, 9, 1));
        JProgressBar health = new JProgressBar(0, 100);
        health.setValue(100);
        health.setStringPainted(true);
        JTextArea log = new JTextArea(4, 20);
        log.setEditable(false);

        JButton hit = new JButton("Take Damage");
        hit.addActionListener(e -> {
            health.setValue(Math.max(0, health.getValue() - 15));
            log.append("Took damage! HP=" + health.getValue() + "\n");
        });
    }
}
```

```

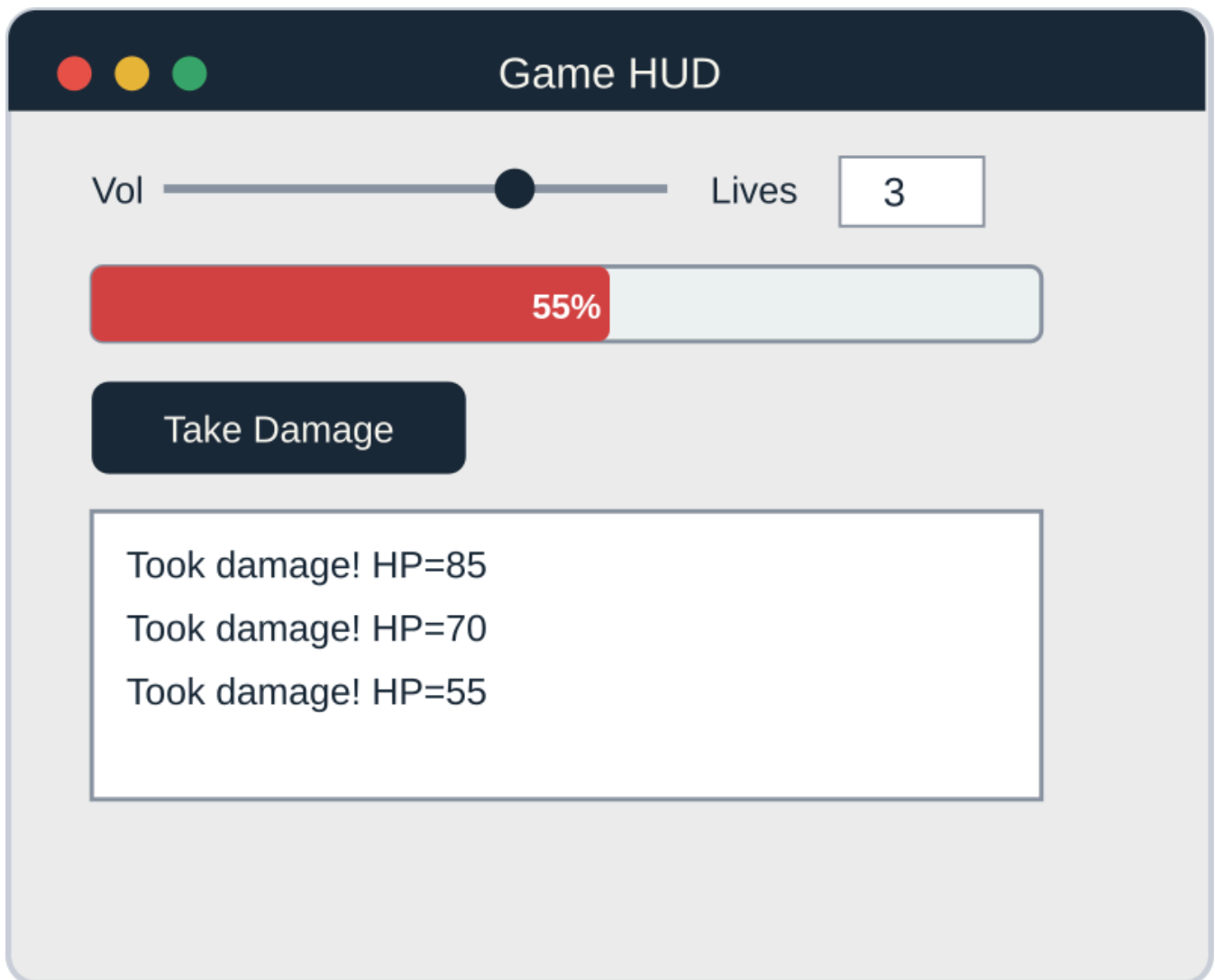
});

panel.add(new JLabel("Vol")); panel.add(volume);
panel.add(new JLabel("Lives")); panel.add(lives);
panel.add(health); panel.add(hit);
panel.add(new JScrollPane(log));

frame.add(panel);
frame.setSize(360, 320);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Expected output:



Try it: add a "Heal" button that adds 10 to health (capped at 100 with `Math.min`) and appends its own log line.

Behind the Scenes: Four Listener Interfaces

Every exercise above used one of four interfaces, each following the same idea from Week 1 — implement one method, Swing calls you back:

Interface	Method	Fires when...	Used with
<code>ActionListener</code>	<code>actionPerformed(ActionEvent e)</code>	the component is "activated"	<code>JRadioButton</code> , buttons
<code>ItemListener</code>	<code>itemStateChanged(ItemEvent e)</code>	the selection state changes	<code>JCheckBox</code> , <code>JComboBox</code>
<code>ListSelectionListener</code>	<code>valueChanged(ListSelectionEvent e)</code>	the highlighted row(s) change	<code>JList</code>
<code>ChangeListener</code>	<code>stateChanged(ChangeEvent e)</code>	a numeric value changes	<code>JSlider</code> , <code>JSpinner</code>

`JProgressBar` fires none of these — nothing selects it, so it never needs a listener at all; it is written to, never read from by the user.

`JCheckBox` and `JRadioButton` actually support both `ActionListener` and `ItemListener` — this lab used `ItemListener` for the checkbox (gives the new state directly) and `ActionListener` for the radio buttons (simpler: "which one fired?").

Submission Checklist

- ☐ `CheckBoxDemo.java` toggles correctly with `ItemListener`
- ☐ `RadioButtonDemo.java` restricts selection to one button — confirmed broken without `ButtonGroup`, then fixed
- ☐ `ComboBoxDemo.java` reads the selected item correctly
- ☐ `ListDemo.java` guards against mid-drag firing with `getValuesAdjusting()`
- ☐ `GameSettings.java` combines all four selection components behind one Start button
- ☐ `TextAreaDemo.java` appends lines without replacing previous ones
- ☐ `ScrollPaneDemo.java` scrolls a 20-item list inside a fixed-size box
- ☐ `SliderDemo.java` shows tick marks and updates only after the drag settles
- ☐ `SpinnerDemo.java` casts `getValue()` correctly to `Integer`
- ☐ `ProgressBarDemo.java` drains on click with the percentage painted on it
- ☐ `GameHud.java` combines all four value/text components behind one button
- ☐ Can name, for each of the nine components, which listener interface it uses (or none)

Next Week

With all nine components covered, Week 3 moves to `javax.swing.Timer` and the game loop — the first moving shape on screen, animated by a repeating timer event rather than a user click. This is the first real step toward the egg-catching game.

Homework / Research Questions

No answers below — work these out and bring them to the next session.

1. Build it. Extend Exercise 5's `GameSettings` panel with a `JSpinner` for "Number of Players" (range 1–4). Wire it into the existing `startBtn` listener so the summary label also reports the player count. Does adding this fourth input require a second listener, or can it be read from inside the same `ActionListener`? Why?

2. Investigate. `JSlider` fires a `ChangeListener` as the user drags it, and its internal `getValueIsAdjusting()` tells you whether the drag is still in progress. `JScrollBar` (the thing that actually appears inside a `JScrollPane`) has almost the exact same API — including its own `getValueIsAdjusting()`. Look up the `JScrollBar` class in the Java documentation and explain: why doesn't this lab's `JScrollPane` exercise ever need to register a listener on it, even though the component clearly has one available?